Proven ID

Customer Onboarding Integration Guide

Version 0.1

This document describes the system components, setup steps, and workflow for integrating Proven ID's identity verification solution into a customer's onboarding process.

1. Overview

Proven ID is Indicio's identity verification (IDV) solution built on verifiable credentials. It enables customers to integrate credential-based identity verification into their onboarding process through two sub-components:

- Proven ID Library — a server-side library installed on the customer's backend
- Proven ID JavaScript — a client-side code snippet embedded in the customer's webpage

This integration guide covers the credential presentation and verification workflow. Credential issuance — the process by which a user acquires verifiable credentials in their Holdr+ wallet — takes place separately within the Holdr+ app and is outside the scope of this document.

2. System Components

2.1 Indicio Components

Component	Description
Proven ID Library	Server-side library installed on the customer's backend. Provides the configuration interface, WebSocket management, webhook handling, and outbound communication with the Proven Agent.
Proven ID JavaScript	Client-side code snippet embedded in the customer's webpage. Manages the user-facing UI for the verification flow: instructions, QR code, deep link, status updates, and result handling.
Proven Agent	Verifiable credential agent software configured for the customer. Manages DIDComm communication with the Holdr+ app, requests credential presentations, and reports results to the customer's backend via webhook. Can be hosted by Indicio or the customer. Typically one instance per customer,

Copyright © 2026 Indicio PBC. All rights reserved.

Component	Description
	though customers with complex or cross-departmental needs may operate multiple agents.
Holdr+ App	A mobile application that contains the user's verifiable credential wallet. The user scans a QR code or taps a deep link to connect with the Proven Agent and present credentials.

2.2 Customer Components

Component	Description
Backend	The customer's server. Hosts the Proven ID Library, maintains WebSocket connections to the frontend, receives webhook calls from the Proven Agent, and applies business logic to verification results.
Webpage	The customer's frontend. Hosts the Proven ID JavaScript snippet, which injects a self-managed block-level UI element into the page to handle all user-facing interactions during the verification flow.

3. System Setup

Complete the following steps before deploying the integration. Steps 1–5 configure the Proven ID Library on the customer's backend via a configuration file shipped with the library. The library includes a sample configuration file with reasonable defaults where applicable; all fields are required unless otherwise noted in the field-level documentation.

3.1 Backend Configuration (Proven ID Library)

1. Install the Proven ID Library on the customer's backend server.
2. Configure WebSocket settings. These control how the library manages persistent WebSocket connections from the Proven ID JavaScript client.
3. Configure the credential schema. Provide the schema ID of the verifiable credential that users will be required to present during onboarding.
4. Configure validation rules. Define the customer's business logic for evaluating credential attribute values received from the Proven Agent.
5. Configure the redirect URL. Specify the URL to which the Proven ID JavaScript will redirect the user upon a successful result, if a redirect is the desired outcome.

3.2 Proven Agent Configuration

6. Configure the webhook in the Proven Agent UI. Set the webhook URL to point to the customer's backend endpoint that will receive presentation results.
7. Generate an API key for webhook authentication. This can be done from the Proven Agent UI. The API key is sent as an HTTP header on all webhook requests from the Proven Agent to the backend, and must be validated by the backend on receipt. For full

Copyright © 2026 Indicio PBC. All rights reserved.

details on API key management and Proven Agent configuration, refer to the Proven Agent documentation.

3.3 Frontend Setup

8. Include the Proven ID JavaScript snippet on the customer's webpage. The snippet injects a block-level container element that sizes and manages all verification UI automatically. The customer should verify that the surrounding page layout renders acceptably around the injected element.

4. Workflow

The verification workflow consists of two phases: Initialization and Presentation. A persistent WebSocket connection between the Proven ID JavaScript and the customer's backend is established during Initialization and maintained through to the end of the Presentation phase.

4.1 Initialization

Step	Actor	Action	Details
1	Proven ID JavaScript	Display instructions	Renders initial instructions and Holdr+ app download links to the user. This ensures the user has the app and an appropriate credential before proceeding.
2	Proven ID JavaScript	Establish WebSocket	Opens a persistent WebSocket connection to the customer's backend. This connection is maintained for the duration of the session.
3	Backend	Send invitation (implicit)	Upon accepting the WebSocket connection, the backend automatically requests a DIDComm out-of-band (OOB) invitation URL from the Proven Agent and returns it to the client via a WebSocket invitation message.
4	Proven Agent	Generate invitation	Creates a DIDComm OOB invitation URL and returns it to the backend.
5	Proven ID JavaScript	Render QR code and deep link	Receives the invitation URL and renders both a scannable QR code and a tappable deep link for the user.

Copyright © 2026 Indicio PBC. All rights reserved.

4.2 Presentation

Step	Actor	Action	Details
1	Holdr+ App	Connect to Proven Agent	The user scans the QR code or taps the deep link rendered in Initialization step 5. This triggers the Holdr+ app to connect to the Proven Agent using the DIDComm OOB invitation URL.
2	Proven Agent	Request credential presentation	Sends a DIDComm presentation request to the Holdr+ app, specifying the credential schema configured in the backend.
3	Holdr+ App	User approves and presents	The user selects a credential matching the requested schema from their wallet and approves the presentation. The Holdr+ app sends the presentation to the Proven Agent via DIDComm.
4	Proven Agent	Verify credential	Verifies the cryptographic integrity and schema compliance of the presented credential.
5	Proven Agent	Send webhook to backend	Sends an authenticated HTTP webhook to the customer's backend containing the presentation result. See Section 5 for the webhook payload structure.
6	Backend	Send processing message	Sends a WebSocket processing message to the Proven ID JavaScript to indicate that the presentation has been received and business logic is being applied.
7	Backend	Validate and process	Applies the customer's configured business rules to the credential attribute values from the webhook payload. Stores any necessary information.
8	Backend	Send result message	Sends a WebSocket result message to the Proven ID JavaScript containing the outcome, any instructions or error details, and optionally a redirect URL.
9	Proven ID JavaScript	Present outcome	Displays instructions or error details to the user, or redirects to the configured URL, based on the result message received from the backend.

4.3 Session Expiration

The backend manages a session timer for each WebSocket connection. A default timeout value is provided in the sample configuration file. If the timer elapses before the presentation flow completes, the backend sends a session_expired WebSocket message to the Proven ID JavaScript.

Copyright © 2026 Indicio PBC. All rights reserved.

Upon receiving a session_expired message, the Proven ID JavaScript hides the QR code and deep link and displays a “Start Again” button. When the user clicks this button, the existing WebSocket connection is reused and the backend implicitly requests a new invitation from the Proven Agent, restarting the flow from Initialization step 3.

Note: A failed verification — where the credential attributes are delivered but the verification status is failed — should be treated as potentially fraudulent. Attribute values in this state are not trustworthy. The appropriate response is left to the customer’s discretion; options include logging the attempt, rejecting the session, and alerting.

5. WebSocket Message Schema

All WebSocket messages exchanged between the backend and the Proven ID JavaScript use a JSON envelope with a type field. The message types and their fields are defined below.

5.1 Message Types

Type	Direction	Purpose
`invitation`	Backend → JS	Delivers the DIDComm OOB invitation URL for QR code and deep link rendering. Sent automatically when the WebSocket connection is established or re-initialized after session expiry.
`processing`	Backend → JS	Notifies the client that the webhook has been received and business logic is being applied. Triggers a loading or processing state in the UI.
`result`	Backend → JS	Delivers the final verification outcome, including status, optional instructions or error details, and optionally a redirect URL.
`session_expired`	Backend → JS	Notifies the client that the session timer has elapsed. Triggers the “Start Again” UI state.

5.2 invitation

Sent by the backend to the Proven ID JavaScript upon WebSocket connection and upon session restart.

```
{ "type": "invitation", "invitation_url": "<DIDComm OOB URL>" }
```


Copyright © 2026 Indicio PBC. All rights reserved.

Field	Type	Present when	Description
`type`	`string`	Always	Fixed value: "invitation"
`invitation_url`	`string`	Always	The DIDComm out-of-band invitation URL to be rendered as a QR code and deep link.

5.3 processing

Sent by the backend to the Proven ID JavaScript after the webhook is received, before business logic processing completes.

```
{ "type": "processing" }
```

Field	Type	Present when	Description
`type`	`string`	Always	Fixed value: "processing"

5.4 result

Sent by the backend upon completion of business logic processing. The status field determines how the Proven ID JavaScript responds.

```
{ "type": "result", "status": "success" | "failed" | "error",
  "code": "<enum string>",
  "message": "<human-readable string>",
  "redirect_url": "<URL>",
  "attributes": { "schema_id": "...", "values": { "<name>": "<value>", ... }
} }
```

Field	Type	Present when	Description
`type`	`string`	Always	Fixed value: "result"
`status`	`string`	Always	"success": credential verified and business rules passed. "failed": credential presented but verification failed or business rules not met. "error": a system or process error occurred.

Copyright © 2026 Indicio PBC. All rights reserved.

Field	Type	Present when	Description
`code`	`string`	Always	An enum identifier for the specific result. Used as display fallback when message is absent.
`message`	`string`	Optional	A human-readable description of the result. When present, this is displayed to the user. When absent, code is displayed as a fallback.
`redirect_url`	`string`	Optional	When present, the Proven ID JavaScript redirects the user to this URL instead of displaying a message.
`attributes`	`object`	success or failed	Contains schema_id (string) and values (object of attribute name–value pairs). Absent on error status. When status is "failed", attribute values are present but must not be trusted.

Note: The Proven ID Library provides preset code and message values for common outcomes. Custom code and message values can also be sent. When a redirect_url is present, it takes precedence over message display. The determination of which outcome to send is made entirely by the customer's backend business logic.

5.5 session_expired

Sent by the backend when the session timer elapses before the presentation flow completes.

```
{ "type": "session_expired" }
```

Field	Type	Present when	Description
`type`	`string`	Always	Fixed value: "session_expired"

6. Webhook Payload

When the Proven Agent completes a presentation attempt, it sends an HTTP POST request to the webhook URL configured in the Proven Agent UI. The request is authenticated using an API key in an HTTP header.

For details on generating and managing API keys, refer to the Proven Agent documentation.

Copyright © 2026 Indicio PBC. All rights reserved.

6.1 Successful or failed presentation

```
{ "schema_id": "<schema ID string>",
  "attributes": { "<attribute name>": "<attribute value>", ... },
  "verification_status": "success" | "failed" }
```

A `verification_status` of `"failed"` indicates that the credential's cryptographic signature or schema compliance could not be verified. Attribute values may still be present in the payload but are not trustworthy and should not be used for business decisions.

6.2 Error or user rejection

```
{ "error": "<description of error or rejection>" }
```

This payload is sent when a system error occurs during the presentation process, or when the user declines to approve the credential presentation in the Holdr+ app.

7. Configuration Reference

The Proven ID Library ships with a sample configuration file containing all required fields and reasonable defaults where applicable. The table below summarizes the top-level configuration areas.

Configuration area	Description
WebSocket settings	Host, port, and path configuration for the WebSocket server. Includes the session timeout duration (default provided in sample config).
Credential schema	The schema ID of the verifiable credential to be requested during the presentation flow.
Validation rules	Customer-defined business logic applied to credential attribute values received from the Proven Agent webhook.
Redirect URL	The URL to which the Proven ID JavaScript redirects the user when a successful result includes a redirect instruction.
Proven Agent connection	The base URL of the Proven Agent instance this backend communicates with.

For detailed field-level documentation, refer to the inline comments in the sample configuration file.

Copyright © 2026 Indicio PBC. All rights reserved.

8. Related Documentation

- Proven Agent documentation — webhook configuration, API key management, agent setup and administration
- Holdr+ app documentation — credential issuance and wallet management
- Proven API documentation — programmatic agent management and API key generation

Copyright © 2026 Indicio PBC. All rights reserved.